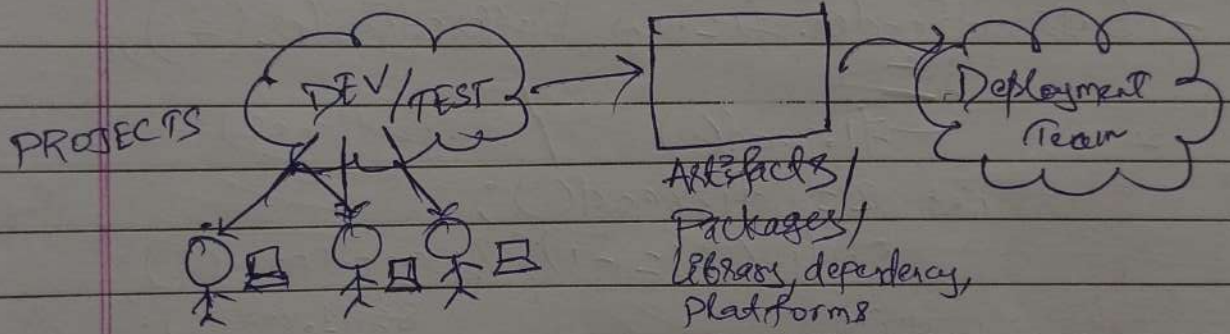
1 Q Docker Definition
2 Q Tradition Dev. Env. Problem
3 Q VMs and solution
4 Q Diff bw VMs & Containers
5 Q Images vs Containers
6 Q Docker Hub

1 → Docker is a linux based platform that is used to create, run and manage containers.
- Containers is an isolated environment in which you can run the application

2 → What the main drawbacks of traditional environment



a Environment for project takes time to create as each machine have separate installation.
b If working on diff projects then there may occur version conflicts.
c Rollback process is not easy as to keep remember of versions (version conflict)
d. Maintain the version / communication team.

VM's adv.
→ Decreases the time for setting up environment.
→ can create 2 VM's environment in 1 machine
→ cloud maintains the versioning like S3 bucket
→ API calls for any resource

Docker

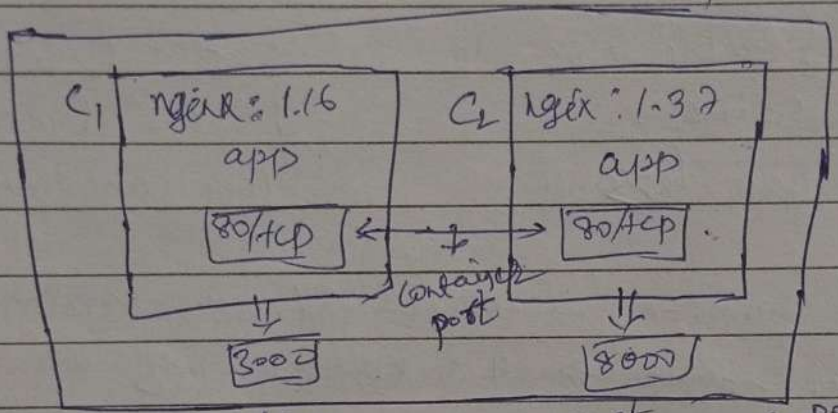
Page No.:
Date: 20/3/25

VM	Containers
• Virtual machine isolates environment at OS level.	Container isolates environment at application level.
• VM are heavy weight (OS, app's)	It is light weight. (app's)
• It takes more time than container.	It takes less time than VM to launch
• You can run ~~more~~ app's.	You can run more app's.

docker version → both client/server version & ek sath
docker --help → all docker commands
docker images → show all images
docker ps → all container in running state
docker ps -a → all " info.
docker pull ~~image~~ nginx → download image from docker hub.
" " nginx : version → for particular version
name
docker (container create) --name nginxCont nginx → creating container
/create
docker start cont name/id → start cont.
docker stop cont. name/id
docker pause " → hybernate state
docker rename " new name
docker unpause " → to unpause
docker inspect cont. id → info of cont
docker logs id/name → all events
docker rm id → delete ~~stopped~~ cont
docker rm -f id → delete forcefully
docker run ~~container~~ name/id → start & run
cont.
↳ on running this command, it will occupy container as it logs event. (it also create new container)

docker run -d --name cont-name nginx
daemon → runs process in bg.
always creates new container.

* If you want to access host on browser then we have to do activity called port mapping



→ port binding localhost → cont-port
docker run -d -p 30001:80 nginx

* if we want to work on the app's shell then we can run the following command.
- docker exec -it cont-id bash
interactive → attach cont. terminal to window terminal

dangling
- docker rmi image-id → to remove image
- docker container prune → rm unused/containers stopped
- docker image prune → remove unused image
- docker system prune -a → delete unused cont, images, ntw, volumes. (everything) • removes all unused not just dangling (⚠
- docker container prune -f → skip conf. & remove.

docker stop $(docker ps -q) → stop all running Dockers cont
list all running cont. ID
docker rm $(docker ps -aq) → list all cont. id (running & stopped)
& remove them.
→ to remove all containers (Running & stopped)
docker stop $(docker ps -q) && docker rm $(docker ps -aq)

Dockerfile

Page No.:
Date: 21/3/25

```
FROM node:latest
RUN mkdir -p /home/app
WORKDIR /home/app
COPY package*.json /home/app
COPY app.js /home/app
COPY ./ → to copy all files.
RUN npm install
EXPOSE 3000
CMD ["node", "app.js"]
```

to build: docker build -t nodejs:vSatya .
(on vscode server; -t = tag; nodejs = image name; vSatya = version; . = work-dir me)

to run on docker:
docker run -d -p 3000:3000 --name nodejscont_name nodejs:vSatya
(nodejs:vSatya = image name)

to publish the image on dockerhub & make public
docker image tag nodejs:vSatya satyamchaudhary/nodejshub:vSatyam
(nodejs:vSatya = image name; satyamchaudhary/nodejshub:vSatyam = account/file name:Version)

docker push satyamchaudhary/nodejshub:vSatyam
→ file to be published on docker hub.

To run services like nodejs
docker run -dit --name nodecont1 node:latest

If Use .env File at Runtime
docker run --env-file .env -p 3000:3000 grad-app

- docker run -it ubuntu → find & download image from docker hub & runs container on creating
  - -it → flag → interactive
- docker run -it -p 1025:1025 -e key=val -e key=val
  - pull & runs
  - -e → env variable passes to docker
  - my-doc-hub/image-name
  - PORT=4000

DOCKER COMPOSE

docker-compose.yml

docker compose up → runs all containers
docker compose down
docker compose up -d

```
version: "3.5"
services:
  postgres:
    image: postgres
    ports: - "5432:5432"
    environment:
      POSTGRES_USER: postgres
      POSTGRES_DB: review
      POSTGRES_PASSWORD: password
  redis
    image: redis
    ports: - "6379":6379"
    environments:
```

- docker network ls → lists
- bridge → default
- docker run -it --network=host busybox
  - no port mapping req., • it is connected on my localhost n/w
  - --network=none → no n/w address
- docker network create -d bridge yt-name → own n/w
- now i can assign created n/w to diff cont. so they can communicate
- docker network inspect yt-name → n/w inspect

Docker Volumes → data copying to host machine

docker run -it -v /c:/Satyam/Download : /home/abc ubuntu
host machine address container

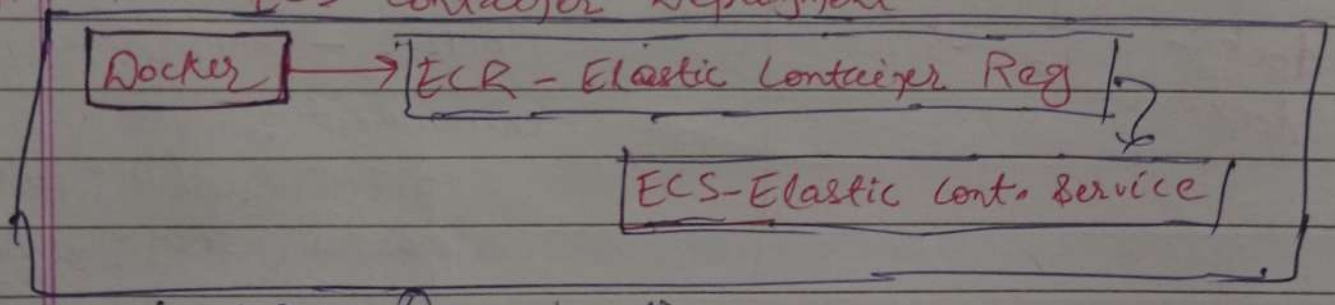
→ volume mapping

Docker Multi-Stage Builds

FROM ubuntu as build
: downloading ts
: converting to js
} build state

FROM node as runner

AWS ECS Container Deployment



Container Orchestration